A PROPOSED DESIGN AND IMPLEMENTATION OF FAMILYVAULT

A MULTI-TENANT, AI-AUGMENTED GENERATIONAL FAMILY PLATFORM

DSN4091-CAPSTONE PROJECT PHASE-I

Phase – I Report

Submitted by

M RUNEET KUMAR (23BAI10051)
RAGHUVENDRA KUMAR (23BAI10563)
ANUSHKA DHIMAN (23BAI10563)
RIYA RATHOD (23BAI10074)
AYUSHMAN GUPTA (23BAI11059)
AYUSH (23BAI10183)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING
VIT BHOPAL UNIVERSITY SEHORE,
MADHYA PRADESH - 466114

September 2026

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE
MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

Certified that this project report titled **"A PROPOSED DESIGN AND IMPLEMENTATION OF FAMILYVAULT – A MULTI-TENANT, AI-AUGMENTED GENERATIONAL FAMILY PLATFORM"** is the bonafide work of **"M RUNEET KUMAR, RAGHUVENDRA KUMAR, ANUSHKA DHIMAN, RIYA RATHOD, AYUSHMAN GUPTA and AYUSH"** who carried out the project work (DSN4091- Capstone Project Phase-I) under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROGRAM CHAIR
Dr Pradeep Kumar Mishra
Senior Assistant Professor (Gr-2),
School of Computing Science Engineering and Artificial Intelligence
VIT BHOPAL UNIVERSITY

PROJECT GUIDE
Dr. Priscilla Dinkar Moyya
Assistant Professor (Gr-2) or other designation
School of Computing Science Engineering and Artificial Intelligence
VIT BHOPAL UNIVERSITY

The DSN4091-Capstone Project Phase-I Viva Voce Examination is held on 06/10/2026.

ACKNOWLEDGEMENT

First and foremost, we would like to thank the Lord Almighty for his presence and immense blessings throughout the project work.

We would like to thank our internal guide **Dr. Priscilla Dinkar Moyya** , for continually guiding and actively participating in our project, and giving valuable suggestions to complete the project works.

We wish to express our heartfelt gratitude to Dr. Pradeep Kumar Mishra, PC-Lead, School of Computing Science Engineering and Artificial Intelligence for much of his valuable support and encouragement in carrying out this work.

We wish to express our heartfelt gratitude to Dr. S Poonkuntran , Dean, School of Computing Science Engineering and Artificial Intelligence for much of his valuable support and encouragement in carrying out this work.

We would like to thank all the technical and teaching staff of the School of Computing Science Engineering and Artificial Intelligence, who extended directly or indirectly all support.

Last, but not least, we are deeply indebted to our parents who have been the greatest support while we worked day and night for the project to make it a success.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Project schedule: Day 0, four sprints and demo day	**13**
4.2	Three visibility tiers enforced by Row Level Security	**15**
4.3	Biometric matching and media inbox workflow	**17**
4.4	FamilyVault system architecture	**18**
4.5	Wireframe of the tree view with person drawer	**19**
5.1	Git branching and release flow	**24**
A.1	Landing page and dashboard shell (desktop and mobile)	**33**
A.2	Family tree canvas with kinship badges	**34**
A.3	Person drawer with gallery, health alerts and legacy letters	**34**
A.4	Media inbox with Accept / Reject actions	**35**
A.5	Emergency Red Alert dossier output	**35**

ABSTRACT

Indian joint families accumulate decades of documents, medical records, property papers, photographs and oral history, yet this heritage usually lives in scattered physical files, personal phones and messaging groups. When a family elder falls ill, a document is needed urgently, or a new generation asks who a relative is, the information is either missing or exposed to the wrong people. This project proposes the design and implementation of FamilyVault, a multi-tenant, AI-augmented generational family platform. FamilyVault combines a secure vault for documents, health records, pets and property, protected by three-tier Row Level Security in PostgreSQL (JOINT_CLAN, NUCLEAR_HOUSEHOLD and PRIVATE_STRICT); a Neo4j kinship graph that computes culturally precise relationship titles such as Chacha, Mama, Bua and Masi while blocking impossible lineage cycles; and a biometric media pipeline that uses InsightFace 512-dimensional embeddings with a 0.85 cosine-similarity threshold to tag family members in group photographs and route uncertain media to a human-approval inbox. A Digital Will and Time Capsule engine with milestone-locked documents and an Emergency "Red Alert" medical dossier extend the platform from storage into legacy preservation and crisis response. The system is built with Next.js, FastAPI, Supabase PostgreSQL, Neo4j AuraDB and Cloudflare R2, and is delivered by a six-member team through four three-day sprints under a contracts-first, pull-request-driven Git workflow. This report presents the problem analysis, related work, requirements, design methodology, technical implementation, verification criteria and future scope of the project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	List of Figures	iv
	Abstract	v
1	**CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE**	**1**
	1.1 Introduction	1
	1.2 Motivation for the Work	1
	1.3 Problem Statement	2
	1.4 Objective of the Work	3
	1.5 Summary	3
2	**CHAPTER-2: RELATED WORK INVESTIGATION**	**5**
	2.1 Existing Approaches/Methods	5
	2.2 Pros and Cons of the Stated Approaches/Methods	6
3	**CHAPTER-3: REQUIREMENT ARTIFACTS**	**8**
	3.1 Introduction	8
	3.2 Hardware and Software requirements	8
	3.3 Specific Project Requirements	9
	3.3.1 Data Requirements	9
	3.3.2 Functional Requirements	10
	3.3.3 Performance and Security Requirements	11
	3.3.4 Look and Feel Requirements	11
	3.4 Summary	12
4	**CHAPTER-4: DESIGN METHODOLOGY AND ITS NOVELTY**	**13**
	4.1 Methodology and goal	13
	4.2 Functional modules design and analysis	14
	4.3 Software architectural designs	17
	4.4 User interface designs	19
	4.5 Summary	20

CHAPTER NO.	TITLE	PAGE NO.
5	**CHAPTER-5: TECHNICAL IMPLEMENTATION & ANALYSIS**	**21**
	5.1 Outline	21
	5.2 Technical coding and code solutions	22
	5.3 Prototype submission	24
	5.4 Summary	25
6	**CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY**	**26**
	6.1 Key implementations outline of the System	26
	6.2 Significant Project Outcomes	27
	6.3 Project applicability on Real-world applications	28
	6.4 Inference	28
7	**CHAPTER-7: CONCLUSIONS AND RECOMMENDATION**	**30**
	7.1 Outline	30
	7.2 Limitation/Constraints of the System	31
	7.3 Future Enhancements	31
	7.4 Inference	32
	APPENDIX A – Screen Shots	33
	APPENDIX B – Coding	36
	REFERENCES	40

CHAPTER 1
PROJECT DESCRIPTION AND OUTLINE

1.1 INTRODUCTION

In many Indian households the family is not a single household but a joint clan: several nuclear families linked across three or more generations through shared ancestry, property and tradition. Over the years such a clan produces a large volume of valuable information, including ancestral deeds, property papers, medical histories, pet vaccination records, photographs, voice recordings and the stories told by its elders. This information is usually scattered across steel almirahs, personal phones and messaging groups, where it is hard to find, easy to lose and impossible to share selectively.

This report proposes the design and implementation of **FamilyVault**, a secure, multi-tenant web platform that works as a generational operating system for a family. FamilyVault stores documents, health records, property and pet information in a vault whose visibility is controlled by three privacy tiers; models the whole clan as a relationship graph that can name every relative the way the family itself would (Chacha, Mama, Bua, Masi, Tauji); and uses face recognition to tag family members in photographs while keeping a human in the loop for every uncertain match. Features such as a Digital Will and Time Capsule engine and an Emergency "Red Alert" medical dossier extend the platform from storage into legacy preservation and crisis response.

Because every member of the six-person development team works with AI coding assistants, standard CRUD operations and basic user interfaces are treated only as the baseline expectation. The value of the project lies in its architecture: tenant isolation across two different databases, culturally precise kinship computation, a single source of truth for identity, and a biometric pipeline with well-defined accuracy boundaries. This report analyses the problem, surveys related work, specifies the requirements, describes the design methodology and technical implementation, and discusses the outcomes, limitations and future scope of the project.

1.2 MOTIVATION FOR THE WORK

The motivation for FamilyVault comes from a set of everyday problems that existing tools address only in part:

- **Fragmented and fragile records.** Ancestral deeds, property documents and medical files are held by different relatives in different places. When one document is urgently needed, nobody is sure who holds it, or whether it survived.
- **A family is not one trust zone.** Ancestral records may reasonably be seen by the entire clan, financial records only by one household, and a personal letter only by its author and a chosen trustee. General-purpose storage tools offer sharing links, but not this layered model of privacy.
- **Kinship vocabulary is cultural.** Mainstream genealogy tools describe relatives with generic English terms such as "uncle" or "aunt". In Indian families a father's younger brother (Chacha), father's elder brother (Tauji) and mother's brother (Mama) are different relations with different roles.
- **Emergencies need information quickly.** Blood group, allergies, vital prescriptions and emergency contacts are rarely available in one place when a family elder is rushed to a hospital.
- **Legacy is time-bound.** Families want to leave letters and documents that open on a future date, such as a child's 18th birthday, or only after a trustee confirms an event. Ordinary storage cannot enforce such conditions.
- **Technical motivation.** The project brings together a relational database, a graph database, object storage and face recognition inside one coherent system, and gives the team experience of professional practices such as branch protection, pull-request review, contract-first integration and penetration testing.

1.3 PROBLEM STATEMENT

Joint families lack a single, secure and culturally aware platform to preserve and share their documents, health information, photographs and relationships across generations. Existing genealogy services concentrate on ancestry research and use Western kinship terms; general cloud storage lacks family-structured access control, lineage awareness and legacy features; and photo applications that recognise faces do not connect the faces to a verified family tree or to a privacy model. As a result, families depend on informal channels that expose private data to the wrong people, lose critical records, and cannot answer simple questions such as "who is this person to me?" or "who may see this document?".

There is therefore a need to design and implement a platform that (i) isolates data between families and between households inside a family, (ii) represents the clan as a validated kinship graph with culturally precise relationship titles, (iii) links media to people through biometric matching with a controlled approval workflow, and (iv) supports emergency access and time-locked legacy content, while keeping identities consistent across every data store.

1.4 OBJECTIVE OF THE WORK

The primary objective of this project is to design and implement FamilyVault as a secure generational family platform. The specific objectives are:

1. Design a multi-tenant relational schema (joint families, nuclear families, users, documents, media contributions) and enforce three visibility tiers, JOINT_CLAN, NUCLEAR_HOUSEHOLD and PRIVATE_STRICT, through PostgreSQL Row Level Security.
2. Implement a vault with create, read, update and delete operations for documents, property, pets and health records.
3. Model the family as a Neo4j graph and compute culturally precise kinship titles using Cypher shortest-path queries, while rejecting impossible lineage cycles and chronological paradoxes.
4. Guarantee a single source of truth for identity by generating each user UUID once in PostgreSQL and reusing it as the Neo4j Person identifier and the biometric association key.
5. Build a multi-person biometric pipeline using InsightFace 512-dimensional embeddings with a shared MATCH_THRESHOLD of 0.85, automatic tagging of recognised members, and an inbox for manual Accept/Reject of uncertain media.
6. Provide the Digital Will and Time Capsule engine with milestone-locked documents and the Emergency “Red Alert” medical dossier endpoint.
7. Develop an interactive Next.js interface with a React Flow family tree, a person drawer, kinship path highlighting, a media inbox and a generational lore timeline with an emergency SOS trigger.
8. Verify the system through automated API tests, a cross-tenant breach simulation against both PostgreSQL and Neo4j, and a biometric boundary test suite, and deploy a demonstrable build with ten seeded users across three generations.

By achieving these objectives, FamilyVault will give a family one trustworthy place for its records and relationships, with privacy and cultural accuracy built into the foundation instead of added later.

1.5 SUMMARY

This chapter introduced FamilyVault, a multi-tenant, AI-augmented platform for preserving the documents, health information, media and relationships of a joint family. The motivation arises from fragmented records, the need for layered privacy, generic kinship terms, emergency information needs and time-locked legacy content. The eight objectives centre on tenant isolation, a validated kinship graph, a controlled biometric workflow and one consistent identity across all data stores. The next chapter reviews existing approaches.

CHAPTER 2
RELATED WORK INVESTIGATION

2.1 EXISTING APPROACHES/METHODS

FamilyVault touches several established product categories and technologies. The approaches that are most relevant to the project are described below.

- **Genealogy and family-tree platforms.** Services such as Ancestry, MyHeritage and FamilySearch let users build trees, attach records and discover relatives. Their strength is research depth, but their focus is historical discovery rather than private, day-to-day family administration, and their relationship labels follow Western conventions.
- **General cloud storage.** Google Drive, Dropbox and OneDrive provide reliable file storage and link-based sharing. They have no concept of a joint clan, a nuclear household or a family tree, so access rules must be managed manually, folder by folder.
- **Government document wallets.** DigiLocker in India stores government-issued documents in digital form. It is valuable for official records but does not cover family photographs, health notes, pet records or family relationships.
- **Personal health record systems.** National and commercial health-record platforms, such as those built around the ABHA health ID, keep individual medical histories. They are designed around the individual patient, not around a family unit with shared emergency access.
- **Digital legacy features.** Account-level tools such as Google's Inactive Account Manager and Apple's Legacy Contact let a designated person receive data after the account holder is gone. They are tied to one vendor account and do not support milestone-based unlocking, such as a future birthday, for specific documents.
- **Face grouping in photo applications.** Consumer photo applications cluster faces and let users name them. The names are free text, are not tied to a verified family graph and the matching has no explicit, auditable threshold or approval workflow.
- **Graph databases for relationship modelling.** Property graphs such as Neo4j represent people and relationships natively and answer questions such as "shortest path between two people" with concise Cypher queries (Robinson, Webber and Eifrem, 2015). This approach is well suited to kinship but needs its own access control.

- **Row Level Security for multi-tenancy.** PostgreSQL and Supabase allow access policies to be attached directly to tables, so that isolation is enforced by the database rather than by application code alone.
- **Face embeddings and cosine similarity.** Modern recognition libraries such as InsightFace convert a face into a fixed-length vector (Deng et al., 2019); two faces are compared by the cosine of the angle between their vectors, and a threshold decides whether they match.

These approaches provide useful building blocks, but none of them combines tiered family privacy, culturally accurate kinship, biometric tagging with human approval, and legacy features in one product. FamilyVault is designed to fill this gap by composing the last three technologies above into a single platform.

2.2 PROS AND CONS OF THE STATED APPROACHES/METHODS

Pros:

- Genealogy platforms offer mature tree-building interfaces and large record collections.
- Cloud storage is reliable, familiar to non-technical relatives and inexpensive.
- Document wallets and health-record systems provide trusted, officially recognised records.
- Graph databases make multi-hop relationship queries simple, fast and readable.
- Row Level Security enforces isolation at the data layer, so a bug in one API route cannot silently expose another tenant's rows.
- Embedding-based face matching is accurate enough to automate the common case and allows an explicit confidence threshold.

Cons:

- Genealogy tools and cloud storage have no family-structured privacy model and no culturally precise kinship terms.
- Each product solves one slice of the problem, so families must juggle several tools and keep identities in sync by hand.
- Legacy features are tied to one vendor and cannot unlock individual documents on future milestones.
- Row Level Security protects only PostgreSQL; a separate graph database needs its own tenant-isolation checks.

- Biometric matching can fail on poor lighting, group photographs, children who grow up or faces that are not detected, so automation alone is not acceptable for uncertain cases.
- Biometric data is sensitive personal data and must be handled with consent and care (Government of India, 2023).

Table 2.1: Indicative category-level comparison of capabilities

Capability	Genealogy platforms	Cloud storage	Photo apps	FamilyVault
Family tree	Yes	No	No	Yes
Culturally precise kinship titles	Limited	No	No	Yes
Tiered family privacy	Partial	Partial (manual)	Partial	Yes (3 tiers)
Document and health vault	Partial	Yes (files only)	No	Yes
Face-based tagging with approval	No	No	Partial	Yes
Milestone-locked legacy documents	No	No	No	Yes
Emergency medical dossier	No	No	No	Yes

The table is a qualitative, category-level comparison intended to position the project and is not a benchmark of specific products.

When these approaches are weighed against the needs of a joint family, the most appropriate strategy is composition: use a relational database with Row Level Security for records, a graph database for lineage, object storage for media and an embedding model for faces, and tie them together with one identifier and explicit contracts between services.

CHAPTER 3
REQUIREMENT ARTIFACTS

3.1 INTRODUCTION

Requirements ensure that FamilyVault is functional, secure and usable by people of all ages in a family. They fall into three groups: functional requirements, which describe what the system does; non-functional requirements, which describe performance, security and reliability; and usability requirements, which describe how the system looks and feels. Because the work is divided among six team members, requirements are also expressed as shared contracts, for example the response format of the tree endpoint and the matching threshold, so that every module can be built and tested independently.

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

Hardware requirements. FamilyVault uses managed cloud services for storage and hosting, so the hardware needed by developers is modest.

Table 3.1: Hardware requirements

Component	Requirement	Purpose
Developer workstation	Multi-core CPU, 8 GB RAM minimum (16 GB recommended), SSD storage	Run the frontend and three FastAPI services locally; CPU-only InsightFace inference
Network	Stable broadband connection	Access to Supabase, Neo4j AuraDB, Cloudflare R2 and GitHub
Test devices	At least one physical smartphone	Mobile viewport testing on a real device, not only a resized desktop browser
Backend host	Managed container hosting (Render or Railway)	Run the FastAPI services in production
Frontend host	Vercel	Serve the Next.js application
Data services	Supabase PostgreSQL, Neo4j AuraDB, Cloudflare R2	Relational data, graph data and media objects

Software requirements. The technology stack is summarised below.

Table 3.2: Software requirements

Layer	Technology	Purpose
Frontend	Next.js (App Router), Tailwind CSS, shadcn/ui	Application shell, dashboards, responsive layout
Tree visualisation	@xyflow/react (React Flow)	Pan/zoom family tree canvas with custom nodes
Core backend	Python, FastAPI	Vault, health, pets, users, will and time capsule, emergency dossier
Graph service	FastAPI, Neo4j driver, Cypher	Kinship calculation, paradox validation, tree endpoint
Biometrics service	FastAPI, InsightFace, NumPy	Face detection, 512-D embeddings, cosine matching
Relational database	Supabase PostgreSQL with Row Level Security	Tenants, users, documents, media contributions
Graph database	Neo4j AuraDB	Person nodes and kinship relationships
Object storage	Cloudflare R2	Photos, video and audio via signed URLs
Testing	Postman, Pytest, Playwright	API, security and end-to-end tests
Collaboration	Git, GitHub (branch protection, Projects board), Slack/Discord	Version control, review and task tracking
Deployment	Render/Railway (backend), Vercel (frontend)	Public hosting of the final build

3.3 SPECIFIC PROJECT REQUIREMENTS

3.3.1 Data Requirements

Table 3.3: Principal data entities

Entity	Store	Key contents
joint_families, nuclear_families	PostgreSQL	The clan and the households inside it; every other record references them
users	PostgreSQL	UUID primary key, name, gender, birth date, role (Admin, Editor, Viewer), family links
documents	PostgreSQL	Vault items with owner, visibility tier, storage key and unlock conditions
media_contributions	PostgreSQL	Uploader, R2 object key, media type, match score and review status
(:Person) node	Neo4j	Same UUID as users.id, gender, birth date, gotra and lineage relationships

Entity	Store	Key contents
Media objects	Cloudflare R2	Photos, videos and audio accessed only through signed URLs
Face embeddings	Biometrics service	512-dimensional vectors associated with a user UUID
Health data	PostgreSQL	Blood group, allergies, vital prescriptions, emergency contacts, pet vaccinations
Shared constants	/contracts	MATCH_THRESHOLD = 0.85 and the mock JSON schemas used for integration
Seed data	backend/scripts/seed.py	Ten dummy users across three generations for repeatable testing

3.3.2 Functional Requirements

- **FR1.** The system shall support three roles (Admin, Editor, Viewer) and authenticate every request.
- **FR2.** The system shall allow authorised users to create, view, update and delete documents, property, pet and health records.
- **FR3.** Each record shall carry one of three visibility tiers and shall be returned only to users permitted by that tier.
- **FR4.** The system shall store time-capsule documents with unlock conditions and release them only when the conditions are met.
- **FR5.** The system shall compile an emergency dossier (allergies, blood group, vital prescriptions, emergency contacts) on demand through POST /api/v1/health/emergency-dossier/{user_id}.
- **FR6.** The system shall expose the family tree through GET /api/graph/tree, including a kinship badge for each member.
- **FR7.** The system shall calculate culturally precise kinship titles and shall reject parent-child links that create a chronological or cyclic paradox.
- **FR8.** The system shall accept image, video and audio uploads through signed URLs and detect every face in an uploaded image.
- **FR9.** The system shall publish a media item automatically only when matches meet MATCH_THRESHOLD, and shall send all other items, including all video and audio, to the inbox.

- **FR10.** Inbox items shall be accepted or rejected by the reviewing member; rejected media shall remain in the sender's own archive.
- **FR11.** The interface shall provide a person drawer (gallery, health alerts, unlocked legacy letters), kinship path highlighting, tree search and a lore timeline with an SOS trigger.
- **FR12.** The system shall provide a seed script that recreates the ten-user, three-generation dataset in PostgreSQL and Neo4j with identical UUIDs.

3.3.3 Performance and Security Requirements

Performance requirements.

- The interface shall remain responsive on both desktop and mobile viewports, including loading states for slow requests.
- The tree canvas shall render a three-generation family smoothly with pan, zoom and search.
- Slow database queries identified during testing shall be optimised before deployment, and API failures shall return handled errors instead of unhandled exceptions.

Security requirements.

- Row Level Security shall be enabled on all tenant tables, and a user shall never be able to read another tenant's rows by using a valid session token.
- Neo4j queries shall be tenant-scoped explicitly, because Row Level Security does not protect the graph database.
- Credentials shall never be committed to the repository; they are kept in .env and .env.local files listed in .gitignore and in a restricted shared credentials document.
- Media shall be accessible only through short-lived signed URLs.
- Changes touching authentication, Row Level Security or biometric matching shall be reviewed by the quality-assurance owner in addition to the normal reviewer.
- The matching threshold shall be defined in exactly one shared constant and imported wherever it is needed.

3.3.4 Look and Feel Requirements

- The interface shall use a clean, modern design built from shadcn/ui components and Tailwind CSS, with consistent spacing and typography.

- Navigation shall be simple enough for older relatives to use: a small number of top-level sections (Tree, Vault, Health, Media Inbox, Lore) and clear labels.
- The tree shall visually distinguish living members, ancestors and family heads, and shall highlight the kinship path between two selected people.
- The emergency SOS control shall always be visible and easy to reach.
- All screens shall work on small mobile viewports without horizontal scrolling.

3.4 SUMMARY

This chapter defined the hardware, software, data, functional, performance, security and look-and-feel requirements of FamilyVault. The requirements place particular emphasis on tenant isolation across both databases, a single identity across all stores, and a single shared matching threshold. They form the basis for the design methodology presented in the next chapter.

CHAPTER 4
DESIGN METHODOLOGY AND ITS NOVELTY

4.1 METHODOLOGY AND GOAL

FamilyVault follows an **agile, sprint-based methodology** adapted to a compressed schedule. After a one-day setup (Day 0), the work is divided into four three-day sprints, followed by a final demonstration day. Each sprint has a clear theme, and every team member has a defined set of branches, actions and acceptance conditions for it. Work is integrated continuously through pull requests, and the stable branch is updated only at the end of a sprint after quality-assurance sign-off.

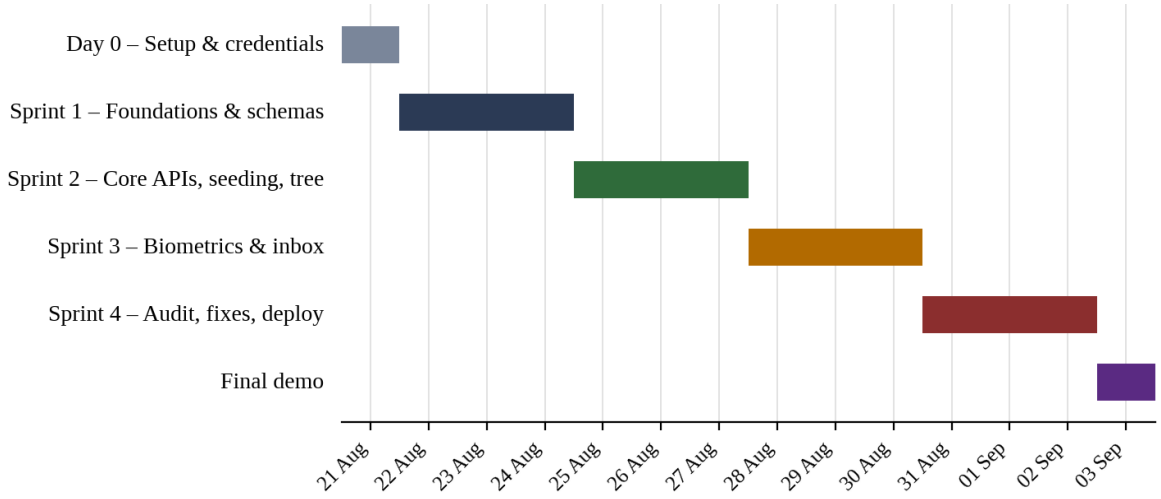


Fig 4.1: Project schedule: Day 0, four sprints and demo day

Three practices make the methodology effective for a team whose work depends on each other:

- **Contracts-first handoff.** The owner of an API publishes a hand-written mock of its response, for example /contracts/graph-tree-response.json, on Day 1 or 2 of a sprint. The consumer builds against the mock immediately and switches to the live endpoint later. This removes idle waiting along the dependency chain Runeet → Riya → Anushka and Riya/Ayushman → Raghuvendra. Any later change to a contract must be announced the same day.
- **Pull-request-driven Git workflow.** Feature branches merge into develop through reviewed pull requests; develop is merged into main only at the end of each sprint and tagged (sprint-1 to sprint-4-release), so main is always deployable.

- **Handoff-centred daily stand-up.** The 15-minute stand-up asks what is about to be handed off, what was merged, and what is blocked, instead of generic status reports.

The **goal** of the methodology is to deliver a demonstrable, secure and integrated FamilyVault build on schedule, while ensuring that each module is independently testable and that identity and privacy rules remain consistent across services. The team structure supports this goal:

Table 4.1: Team roles and ownership

Member	Role	Primary ownership
Runeet	Repository owner; backend and database architect	GitHub repository and branch protection, FastAPI core, Supabase schema, three-tier RLS, seed script, deployment, sprint merges
Riya	Graph infrastructure; kinship and lineage engine	Neo4j AuraDB, Cypher constraints, shortest-path kinship, paradox validation, /api/graph/tree
Raghuvendra	Frontend lead	Next.js scaffold, landing page, dashboard shell, nuclear-family dashboard, media inbox UI
Anushka	Frontend: tree canvas	React Flow canvas, custom nodes, live tree data, search, person drawer, path highlighting
Ayushman	Media infrastructure; biometrics	Cloudflare R2, signed-URL uploads, InsightFace embeddings, cosine matching, inbox routing
Ayush	Quality assurance; security testing	Testing matrix, CRUD and RLS tests, biometric edge cases, cross-database ID check, final penetration audit

4.2 FUNCTIONAL MODULES DESIGN AND ANALYSIS

FamilyVault is divided into functional modules that map to the three backend services and the frontend. Each module has a single owner and a published contract.

Table 4.2: Functional modules

No.	Module	Responsibility	Owner
M1	Tenant and identity	Joint families, nuclear families, users, roles, UUID source of truth	Runeet
M2	Vault	CRUD for documents, property, pets and health records under RLS	Runeet
M3	Digital Will and Time Capsule	Milestone-locked documents with unlock_conditions	Runeet
M4	Emergency Red Alert dossier	Compile allergies, blood group, prescriptions and contacts	Runeet

No.	Module	Responsibility	Owner
M5	Kinship and lineage engine	Culturally precise relationship titles from graph paths	Riya
M6	Graph paradox validation	Block impossible age and cyclic-ancestry links	Riya
M7	Biometric pipeline	Face detection, 512-D embeddings, matching, auto-tagging	Ayushman
M8	Media inbox	Accept/Reject workflow for uncertain media	Ayushman, Raghuvendra
M9	Tree canvas and person drawer	Interactive tree, search, path highlighting, drawer	Anushka
M10	Dashboards and lore timeline	Nuclear-family views, memory feed, SOS trigger	Raghuvendra
M11	Security and quality	Testing matrix, penetration tests, boundary tests	Ayush

M1 and M2: Three-tier Row Level Security

Tenant isolation is enforced by the database. Every record carries a visibility tier, and the Row Level Security policy compares the tier with the caller's joint family, nuclear family and identity.

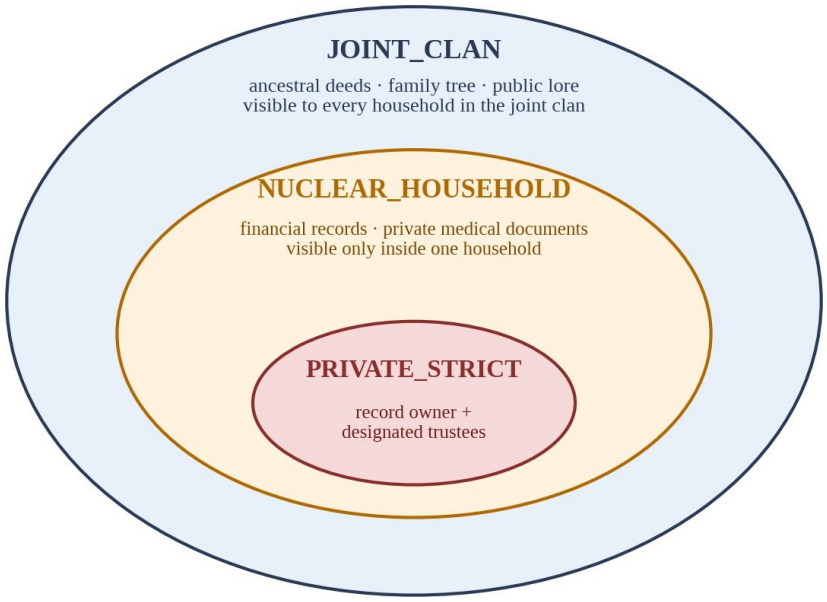


Fig 4.2: Three visibility tiers enforced by Row Level Security

- **JOINT_CLAN** records, such as ancestral deeds, the family tree and public lore, are visible to all nuclear households in the clan.

- **NUCLEAR_HOUSEHOLD** records, such as financial records and private medical documents, are visible only inside the owning household.
- **PRIVATE_STRICT** records are visible only to the creator and to trustees designated by the creator.

Because Row Level Security protects only PostgreSQL, every Neo4j query is also scoped by tenant explicitly, and the penetration tests check both stores separately.

M3 and M4: Time capsule and emergency dossier

A document may carry unlock_conditions, for example a future timestamp such as a child's 18th birthday or a trustee-verified approval. Until the conditions are satisfied the document is hidden from everyone except its owner. The emergency dossier endpoint, POST /api/v1/health/emergency-dossier/{user_id}, compiles allergies, blood group, vital prescriptions and emergency contacts into a single exportable summary that can be produced instantly in a crisis.

M5 and M6: Kinship and lineage engine

Riya's service stores each member as a (:Person) node keyed by the shared UUID, with parent and spouse relationships. To name a relationship, the service finds the shortest path between two people and converts the sequence of hops (up to a parent, down to a child, across a spouse), together with gender, birth order and side of the family, into a title such as Chacha, Tauji, Mama, Bua or Masi, with a paternal or maternal lineage marker. Before a parent link is created, a paradox check rejects a child who is not younger than the parent, a person who would become their own ancestor, and any cyclic ancestry.

M7 and M8: Biometric pipeline and media inbox

When media is uploaded to Cloudflare R2 through a signed URL, the biometrics service detects every face in an image, generates a 512-dimensional embedding for each, and compares it with the stored embeddings of family members using cosine similarity. The decision rule is deliberately conservative: matches at or above the shared MATCH_THRESHOLD of 0.85 may be published automatically and linked to the matching (:Person) nodes, while everything else (scores below the threshold, images with no detectable face, and all video and audio) is placed in the inbox, where the member concerned accepts or rejects it. In the reference design, an image is auto-published only when every detected face clears the threshold. A rejected item is not deleted; it remains in the sender's own archive.

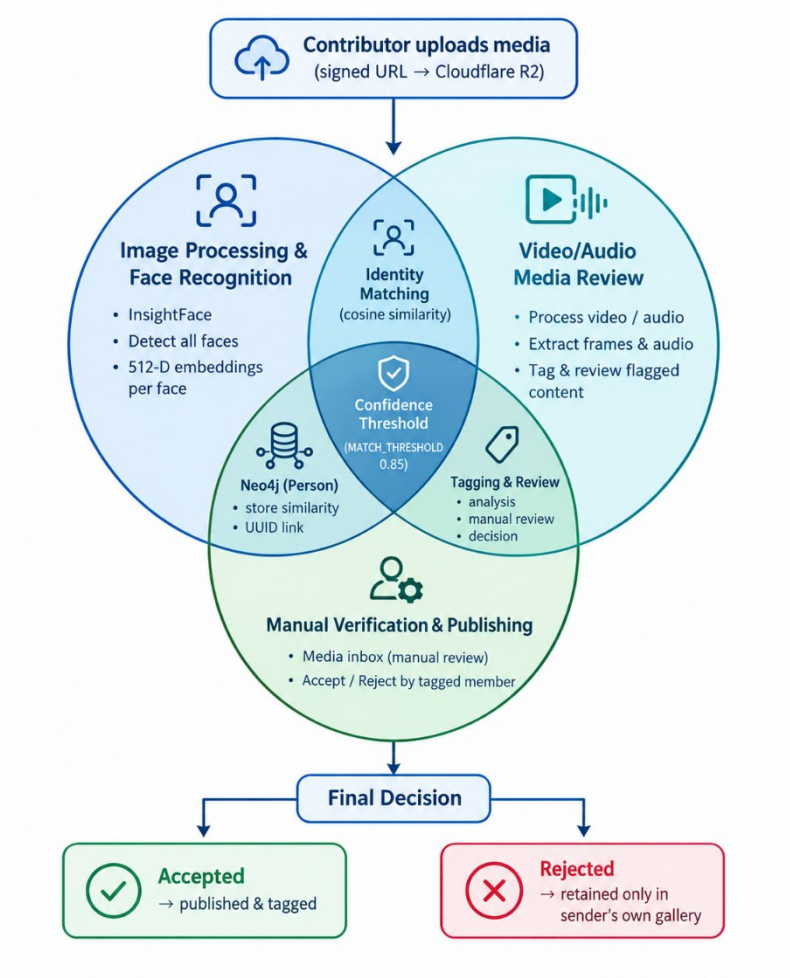


Fig 4.3: Biometric matching and media inbox workflow

M9 and M10: Frontend experience

The frontend is organised around the family tree. Selecting a person opens a drawer containing their photo gallery, health alerts and unlocked legacy letters, and selecting two people highlights the kinship path between them. The nuclear-family dashboard presents personal records, health reminders and pet vaccination timelines, and the lore timeline offers an interactive memory feed of images, audio and video together with an emergency SOS trigger.

4.3 SOFTWARE ARCHITECTURAL DESIGNS

FamilyVault uses a three-tier architecture with three independent backend services. The presentation tier is a Next.js application. The service tier consists of the core backend, the graph

service and the biometrics service, each built with FastAPI and each deployable on its own. The data tier consists of Supabase PostgreSQL, Neo4j AuraDB and Cloudflare R2.

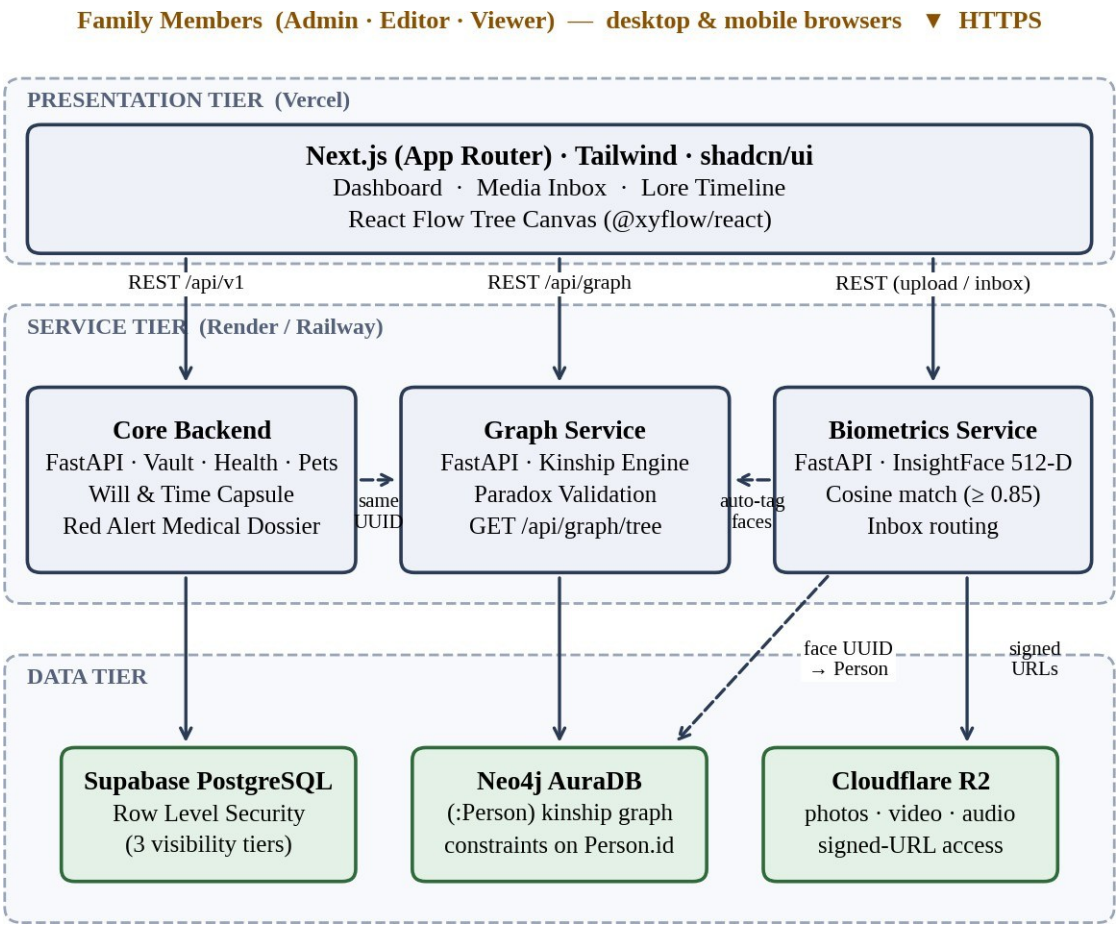


Fig 4.4: FamilyVault system architecture

The main architectural decisions are as follows:

- **Polyglot persistence.** Relational data (tenants, documents, permissions) lives in PostgreSQL, relationships live in Neo4j, and large binary media lives in R2. Each store is used for what it does best.
- **Separate services.** The graph and biometric services are separate from the core backend so that they can be developed, tested and scaled independently by different owners.
- **One identity.** The UUID generated once in PostgreSQL users.id is reused as the Neo4j (:Person {id}) value and as the key for biometric embeddings. It is never regenerated independently, which prevents drift between stores.

- **Shared contracts.** The /contracts folder holds schema.sql, graph-tree-response.json, media-match-response.json and MATCH_CONSTANTS.json. These files define the boundaries between services and between backend and frontend.
- **Signed-URL media access.** The browser uploads to and downloads from R2 using short-lived signed URLs generated by the biometrics service, so media is never exposed publicly.
- **Backend-first deployment.** The backend is deployed and verified reachable before the frontend is pointed at it.

The repository mirrors this architecture: /frontend, /backend, /graph-service, /biometrics, /contracts, /tests and /docs, with /contracts acting as the shared agreement between owners.

4.4 USER INTERFACE DESIGNS

The interface is designed around three goals: clarity for older users, speed of navigation, and visibility of family structure. The top navigation contains Tree, Vault, Health, Media Inbox and Lore, with an SOS control that is always visible. A landing page introduces the product, and the dashboard shell hosts every other screen.

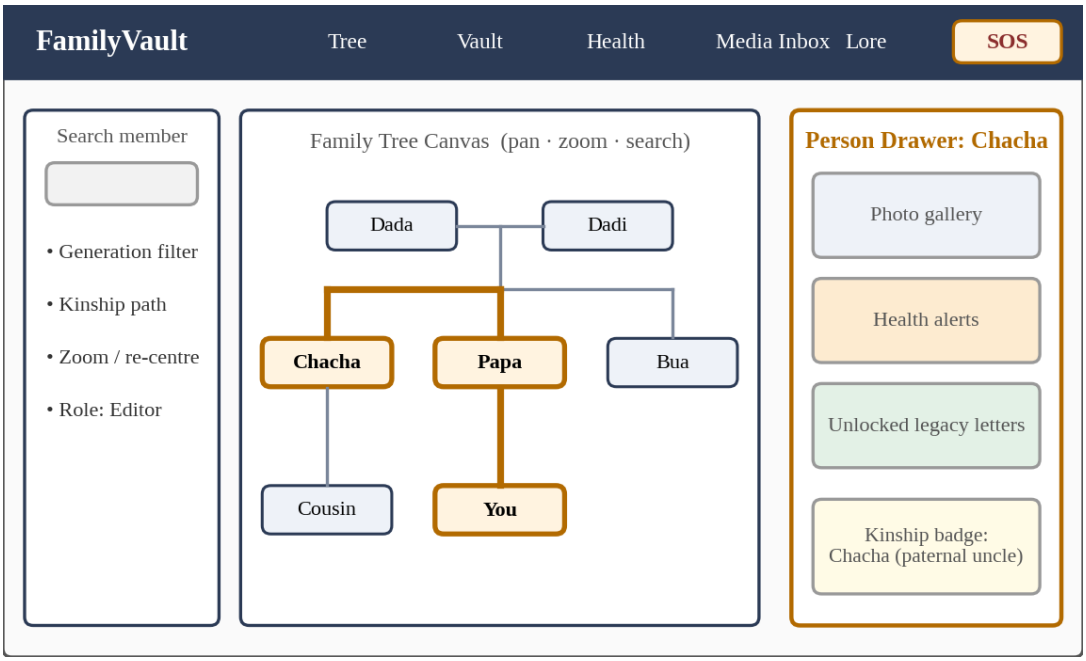


Fig 4.5: Wireframe of the tree view with person drawer

- **Tree canvas.** A pan-and-zoom canvas with custom nodes for living members, ancestors and family heads; a search box centres the view on any member; zoom limits prevent users from getting lost; edges are highlighted along a selected kinship path.
- **Person drawer.** Slides open on node selection and shows the photo gallery, health alerts, unlocked legacy letters and the kinship badge relative to the signed-in user.
- **Nuclear-family dashboard.** Personal records, health reminders and pet vaccination timelines for the household.
- **Media inbox.** A queue of pending media with Accept and Reject actions and a profile preview that shows the kinship badge of the person tagged.
- **Lore timeline.** A chronological feed of images, audio and video memories with an emergency SOS trigger.
- **Responsiveness.** All layouts are tested on desktop and on a real mobile viewport before a pull request is raised.

4.5 SUMMARY

This chapter presented the sprint-based, contracts-first methodology, the eleven functional modules, the three-tier architecture and the user-interface design. The design is novel in the following respects:

- **Three-tier privacy for a family**, combining clan-wide, household-only and owner-and-trustee visibility in one Row Level Security model.
- **Culturally precise kinship** computed from graph paths, with paternal and maternal lineage, instead of generic English relationship labels.
- **Graph paradox validation** that prevents impossible ages and cyclic ancestry from entering the family graph.
- **A conservative biometric workflow** with an explicit, single-source 0.85 threshold, human approval for uncertain media and retention of rejected media in the sender's archive.
- **Legacy features** (time capsule and digital will) and a **one-call emergency dossier** within the same platform.
- **Cross-store integrity**, enforced by a single UUID and verified by an automated cross-database check.

CHAPTER 5
TECHNICAL IMPLEMENTATION & ANALYSIS

5.1 OUTLINE

Implementation begins with a one-time setup on Day 0 (21 August 2026) and proceeds through four sprints. On Day 0, Runeet creates the private repository with main and develop branches, enables branch protection (at least one approving review, no direct pushes), adds all collaborators, pushes the folder structure and CONTRIBUTING.md, and provisions the Supabase project. Riya provisions Neo4j AuraDB, Ayushman creates the R2 bucket and tokens, Ayush creates the Kanban board and the testing channel, and Raghuvendra pushes the Next.js scaffold. All credentials are placed in a restricted shared document and never in the repository.

Table 5.1: Sprint-wise implementation outline

Sprint	Dates	Theme	Principal deliverables
1	22–24 Aug	Foundations, schemas and scaffolding	Database DDL and RLS policies; Neo4j constraints; dashboard shell and landing page; tree canvas base; R2 signed-URL upload; testing matrix
2	25–27 Aug	Core domain APIs, seeding and tree canvas	Vault CRUD and seed script; Cypher shortest-path and /api/graph/tree; nuclear dashboard; live tree data; face-embedding generator; CRUD and RLS penetration tests
3	28–30 Aug	Biometric pipeline and inbox workflow	Cosine match and routing at 0.85; media inbox UI; tree search and polish; Postgres–Neo4j UUID verification; biometric edge-case tests
4	31 Aug – 2 Sep	Final audit, bug fixes and deployment	Full penetration audit with P0/P1/P2 issues; frontend fixes; query optimisation and exception handling; backend then frontend deployment; release tag
Demo	3 Sep	Final demonstration	Live walkthrough of every role on ten seeded accounts; P0 hotfix flow

The elevated architecture and feature standards (three-tier RLS, time capsule, emergency dossier, culturally precise kinship, paradox validation, multi-person biometrics, person drawer, path highlighting, lore timeline and the penetration and boundary suites) are layered onto this schedule as the team mandate for what “done” means in each owner’s area.

5.2 TECHNICAL CODING AND CODE SOLUTIONS

Representative listings of the core logic are provided in Appendix B. The principal solutions are described below.

Setting up the development environment

- **Repository and tooling.** A private GitHub repository "familyvault" with main and develop branches and protection on both. Folders: /frontend, /backend, /graph-service, /biometrics, /contracts, /tests and /docs.
- **Secrets.** .env.local (frontend) and .env (backend, graph-service, biometrics), all listed in .gitignore from the first commit.
- **Services.** Supabase PostgreSQL project, Neo4j AuraDB instance and a Cloudflare R2 bucket with API tokens.

Backend and database (Runeet)

- Write the DDL for joint_families, nuclear_families, users, documents and media_contributions, and deploy it to Supabase (Listing B.2).
- Enable Row Level Security on each table with policies for the three visibility tiers, tested on two dummy tenants.
- Implement CRUD endpoints for the document vault, property, pets and health records, the time-capsule logic and the emergency dossier endpoint (Listing B.6).
- Write backend/scripts/seed.py to create ten users across three generations so that the test suite can recreate a clean state before each cycle.

Graph service (Riya)

- Create a uniqueness constraint on (:Person {id}) and the initial Cypher schema script (Listing B.3).
- Implement shortest-path queries and the hop-to-title mapping for Chacha, Tauji, Mama, Bua and Masi (Listing B.4), with Gotra and paternal/maternal lineage attributes.
- Implement the paradox guard and expose GET /api/graph/tree, keeping /contracts/graph-tree-response.json in step with the real response.

Biometrics service (Ayushman)

- FastAPI controllers that issue signed upload URLs for R2.
- InsightFace integration that produces a 512-dimensional embedding per detected face on upload.

- Cosine-similarity matching against MATCH_THRESHOLD loaded from /contracts/MATCH_CONSTANTS.json, with routing to auto-publish or inbox (Listings B.1 and B.5).

Frontend (Raghuvendra and Anushka)

- Next.js App Router project with Tailwind and shadcn/ui; Anushka adds @xyflow/react on top of the same scaffold to avoid a second project and a painful merge.
- Tree canvas that reads the contract mock first and switches to the live endpoint through an environment flag (Listing B.7); custom node components for living members, ancestors and family heads.
- Nuclear-family dashboard, media inbox with Accept/Reject calling the biometrics endpoints, person drawer, kinship path highlighting and lore timeline.

Testing and security (Ayush)

- Postman and Pytest suites for the CRUD routes, runnable with a single documented command.
- Penetration tests that try to read one tenant's data using another tenant's session token, against both PostgreSQL and Neo4j.
- Biometric boundary tests for group photographs, zero-face uploads and the 0.849 versus 0.850 threshold edge, plus a cross-database script that flags any users.id without a matching Person.id.

Version control and collaboration

Branches follow the pattern feature/<firstname>-<description> or fix/<firstname>-<description>. Commits use conventional prefixes (feat, fix, chore, test, docs). Every pull request targets develop, carries a one-line description of what changed and which contract it affects, is reviewed on the same working day and is squash-merged before the branch is deleted. Changes touching authentication, RLS or biometric matching also require review by Ayush.

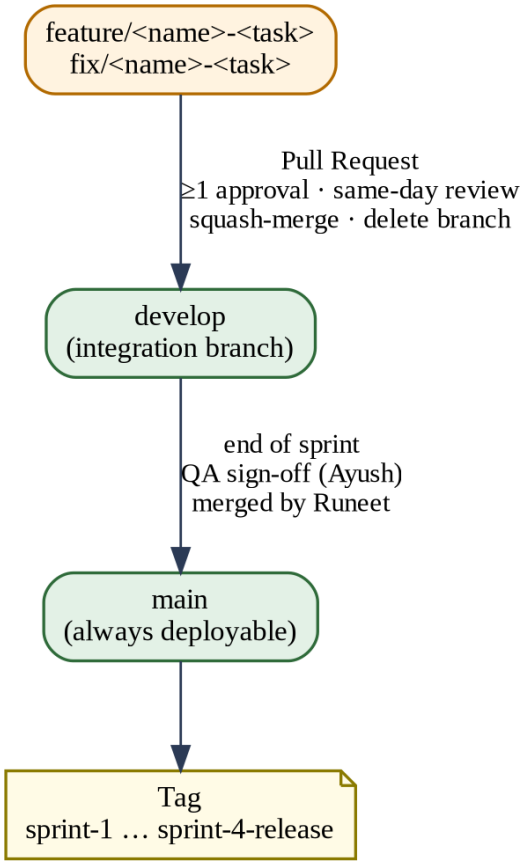


Fig 5.1: Git branching and release flow

Table 5.2: Reviewer pairing

Author	Reviewer	Reason
Runeet	Riya	Schema and ID-synchronisation logic overlap
Riya	Runeet	Schema and ID-synchronisation logic overlap
Raghuvendra	Anushka	Both work inside /frontend
Anushka	Raghuvendra	Both work inside /frontend
Ayushman	Ayush	Security- and biometric-sensitive code
Ayush	Owner of the module under test	Test suites are reviewed by the code owner

5.3 PROTOTYPE SUBMISSION

The Phase-I prototype is a web application made of the Next.js frontend and three FastAPI services connected to Supabase, Neo4j AuraDB and Cloudflare R2. Its scope is defined by the

ten-user, three-generation seed dataset and the three roles (Admin, Editor, Viewer). The prototype demonstrates the following flows:

1. Sign in as a user of a given role and see the dashboard for the user's own household.
2. Create and retrieve vault records at each of the three visibility tiers, and confirm that a user from another household or another clan cannot read them.
3. Open the family tree, search for a member, and read the kinship badge (for example Chacha or Mama) calculated from the graph.
4. Upload a family photograph, observe faces detected and matched against the threshold, and see the result either auto-published or placed in the media inbox.
5. Accept or reject an inbox item and confirm that rejected media remains in the sender's archive.
6. Generate the emergency dossier for a member and view a time-capsule document before and after its unlock condition.

Screenshots of these flows are included in Appendix A. The prototype is deployed with the backend first and the frontend second, and both deployed URLs are confirmed to be reachable and connected before the demonstration.

5.4 SUMMARY

The technical implementation of FamilyVault can be summarised as follows:

- **Database design.** PostgreSQL with Row Level Security enforces three visibility tiers and tenant isolation; the schema and policies are the first deliverable of Sprint 1.
- **Graph and kinship.** Neo4j with Cypher stores the family and computes culturally precise titles, protected by a paradox guard.
- **Biometrics.** InsightFace embeddings and cosine matching at a single shared threshold of 0.85 route media to auto-publish or to a human-reviewed inbox.
- **Frontend.** Next.js, Tailwind, shadcn/ui and React Flow provide the dashboard, tree, person drawer, media inbox and lore timeline.
- **Process.** Contracts-first integration, protected branches, same-day pull-request review and sprint-end tagging keep six contributors' work consistent.
- **Quality.** Automated API tests, a cross-tenant breach simulation, boundary tests and a final penetration audit verify the security and biometric requirements.

CHAPTER 6
PROJECT OUTCOME AND APPLICABILITY

6.1 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

- **Tenant and identity management:**
  - Joint-family and nuclear-family tenants with Admin, Editor and Viewer roles.
  - A UUID generated once in PostgreSQL and reused as the Neo4j Person identifier and biometric key.
- **Privacy and vault:**
  - CRUD for documents, property, pets and health records.
  - Row Level Security with JOINT_CLAN, NUCLEAR_HOUSEHOLD and PRIVATE_STRICT tiers.
- **Legacy and emergency features:**
  - Digital Will and Time Capsule documents with unlock conditions (timestamps or trustee-verified approvals).
  - Emergency Red Alert dossier compiled by a single API call.
- **Kinship and lineage:**
  - Cypher shortest-path queries converted into culturally precise titles with paternal or maternal lineage.
  - Paradox validation that blocks child-older-than-parent links and cyclic ancestry.
- **Biometrics and media:**
  - Signed-URL uploads to Cloudflare R2; multi-person face detection with 512-D InsightFace embeddings.
  - Cosine matching at MATCH_THRESHOLD = 0.85, automatic tagging to graph nodes and a media inbox for everything uncertain.
- **Frontend experience:**
  - Interactive tree with search, zoom limits, custom nodes, person drawer and kinship path highlighting.
  - Nuclear-family dashboard, media inbox and generational lore timeline with SOS trigger.
- **Quality and security:**

- Automated CRUD tests, cross-tenant breach simulation on PostgreSQL and Neo4j, biometric boundary suite and a cross-database ID check.
- Prioritised (P0/P1/P2) issue tracking and a final sign-off checklist before release.

These key implementations form the core of the system and together provide a secure, culturally aware platform for family records, relationships and memories.

6.2 SIGNIFICANT PROJECT OUTCOMES

- **Privacy by construction:** access rules sit in the database, so an application bug cannot by itself expose another family's data.
- **Cultural accuracy:** relatives are named as the family would name them, which makes the tree immediately understandable to elders.
- **Consistent identity:** one UUID across relational, graph and biometric data removes a whole class of synchronisation errors.
- **Safe automation:** a single, shared threshold and a human-reviewed inbox let the system automate clear matches without silently mis-tagging uncertain ones.
- **Preparedness:** the emergency dossier turns scattered medical details into one document available in seconds.
- **Continuity across generations:** time capsules and the lore timeline preserve letters, stories and recordings for future family members.
- **Engineering maturity:** protected branches, contracts, same-day reviews and sprint tagging let six contributors integrate reliably in a short schedule.

The outcomes are verified against the acceptance criteria below, which are derived from the sprint merge conditions and the testing matrix.

Table 6.1: Verification criteria for the principal outcomes

No.	Criterion	Method	Expected result
V1	Tenant isolation in PostgreSQL	RLS demo on two dummy tenants; Ayush's cross-tenant token test	Another tenant's rows are never returned
V2	Tenant isolation in Neo4j	Separate cross-clan graph query test	Queries are tenant-scoped; no foreign nodes returned
V3	Visibility tiers	Read the same record as clan member, household member, non-owner and trustee	Each tier behaves as specified

No.	Criterion	Method	Expected result
V4	Kinship correctness	Tree endpoint for at least two test families	Correct titles (e.g. Chacha, Mama) returned
V5	Paradox validation	Attempt child-older-than-parent and cyclic links	Both are rejected
V6	Embedding size	Upload a test photo	A 512-dimensional vector is produced
V7	Threshold boundary	Scores of 0.849 and 0.850 against MATCH_THRESHOLD	0.849 goes to the inbox; 0.850 can auto-publish
V8	Biometric edge cases	Group photo, zero-face image, rejected media	All faces handled; zero-face goes to inbox; rejected media stays in sender's archive
V9	Identity consistency	Cross-database script on ten seeded users	Every users.id has a matching Person.id; no orphans
V10	Responsive UI	Run through on desktop and a real mobile viewport	No layout breakage; loading states shown

Measured results of these tests are to be recorded in the project's testing matrix and attached to the final submission.

6.3 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

Although designed around the Indian joint family, the concepts behind FamilyVault apply to many settings:

- **Family record-keeping and estate preparedness:** a structured, private home for deeds, property papers and wills, with time-locked release to the right person.
- **Elder care and emergency response:** the Red Alert dossier can be shown to hospital staff and shared with caregivers who are far away.
- **Cultural heritage and community archives:** communities, trusts and societies can preserve lineage, oral history and rituals with accurate regional kinship vocabulary.
- **Family businesses:** succession planning needs both a reliable family tree and carefully tiered access to financial and ancestral documents.
- **Photo and memory management:** the biometric pipeline with human approval is a pattern for any private archive that must tag people responsibly.

- **Multi-tenant SaaS design:** the three-tier Row Level Security model, with an explicit graph-store check, can be reused wherever data must be shared at several trust levels.

These examples show that the core idea of a privacy-aware, relationship-aware platform has wide applicability, provided that the kinship vocabulary and consent practices are adapted to each community.

6.4 INFERENCE

From the requirements, implementation and verification criteria it can be inferred that a layered architecture built from a relational database with Row Level Security, a graph database, object storage and an embedding model is a suitable foundation for a generational family platform. The key design choices are those that reduce risk: a single identity, a single shared threshold, human review of uncertain biometric matches, separate tenant-isolation tests for each database, and contracts-first integration. Together they allow a small team to build a feature-rich system on a compressed schedule without compromising privacy.

CHAPTER 7
CONCLUSIONS AND RECOMMENDATION

7.1 OUTLINE

This project addressed the lack of a secure, culturally aware platform for preserving the records, relationships and memories of a joint family. The design and implementation of FamilyVault achieve the following:

- A multi-tenant data model with three-tier Row Level Security and a single UUID identity across PostgreSQL, Neo4j and the biometric pipeline.
- A kinship graph that produces culturally precise relationship titles and rejects impossible lineages.
- A biometric workflow with multi-person detection, a single 0.85 matching threshold, automatic tagging and a manual approval inbox.
- Digital Will and Time Capsule documents and an Emergency Red Alert dossier.
- An interactive Next.js interface comprising a tree canvas, person drawer, kinship path highlighting, media inbox and lore timeline.
- A repeatable engineering process: contracts-first integration, protected branches, pull-request review, sprint tagging and a dedicated security-testing role.

Through the work we learned several lessons:

- Agreeing on shared identifiers and constants on Day 0 is far cheaper than repairing drift later.
- Publishing a mock contract early allows dependent teammates to start immediately and avoids idle time.
- Row Level Security protects only one database; every additional data store needs its own isolation checks.
- Security testing should start while features are being built, and findings should be logged as individual, prioritised issues so that owners can fix them in parallel.
- Biometric automation must be bounded by explicit thresholds and human review.

Areas for improvement include broader kinship coverage, stronger consent management for biometric data and more extensive performance testing with larger families. Overall, the

project gave the team valuable experience in designing and building a secure, multi-service application, and a solid base on which to continue.

7.2 LIMITATIONS/ CONSTRAINTS OF THE SYSTEM

- **Dataset scale:** development and testing use ten seeded users over three generations; behaviour with very large clans has not been validated.
- **Kinship coverage:** the first release focuses on a core set of North Indian relationship terms (Chacha, Tauji, Mama, Bua, Masi and related lineage markers); other regional vocabularies and complex step or half relationships need additional rules.
- **Biometric limits:** matching accuracy depends on image quality, pose and lighting, and may degrade for children who age or for partially visible faces; CPU-only inference limits throughput.
- **Separate isolation layers:** RLS covers PostgreSQL only, so Neo4j isolation relies on tenant-scoped queries and testing.
- **Schedule:** a compressed, thirteen-day build means some features reach the prototype level rather than full production hardening.
- **Managed-service limits:** the quotas and performance tiers of the hosting, database and storage services constrain scale and availability.
- **Regulatory and consent constraints:** biometric and health data are sensitive personal data and must be handled in line with applicable data-protection law, including informed consent from every person whose face or records are stored.
- **Platform scope:** the first release is a responsive web application, not a native mobile app.

7.3 FUTURE ENHANCEMENTS

- **Mobile applications:** native or installable apps with offline access to the emergency dossier.
- **Multi-language support:** Hindi and other Indian languages, so that elders can use the platform comfortably.
- **Wider kinship vocabularies:** regional and community-specific relationship titles, step and half relations, and in-law relationships.

- **Stronger privacy for the most sensitive records:** end-to-end encryption for PRIVATE_STRICT items and a consent and audit log for biometric use.
- **Smarter media handling:** liveness checks, periodic embedding refresh as children grow, and automatic transcription of audio memories.
- **Document intelligence:** OCR and automatic classification of uploaded documents, with reminders for renewals and expiry dates.
- **Legacy workflows:** notifications for time-capsule unlocks and a guided trustee-verification flow.
- **Integrations:** government document wallets and health-ID services, subject to their policies and the user's consent.
- **Operations:** observability, rate limiting, automated backups and load testing for larger clans.

7.4 INFERENCE

As per the conclusion, FamilyVault is an effective approach to the problem of preserving and sharing family records, relationships and memories securely across generations. Its main strengths are database-enforced privacy, culturally accurate kinship, a safe biometric workflow and legacy and emergency features in one platform. The limitations identified above, particularly dataset scale, kinship coverage and consent management, define a clear path for future work.

Overall, the project demonstrates how careful architecture and disciplined team practice can turn a sensitive, human problem into a secure and usable system, and it highlights the importance of continuous testing and improvement as the platform grows.

APPENDIX A – SCREENSHOTS

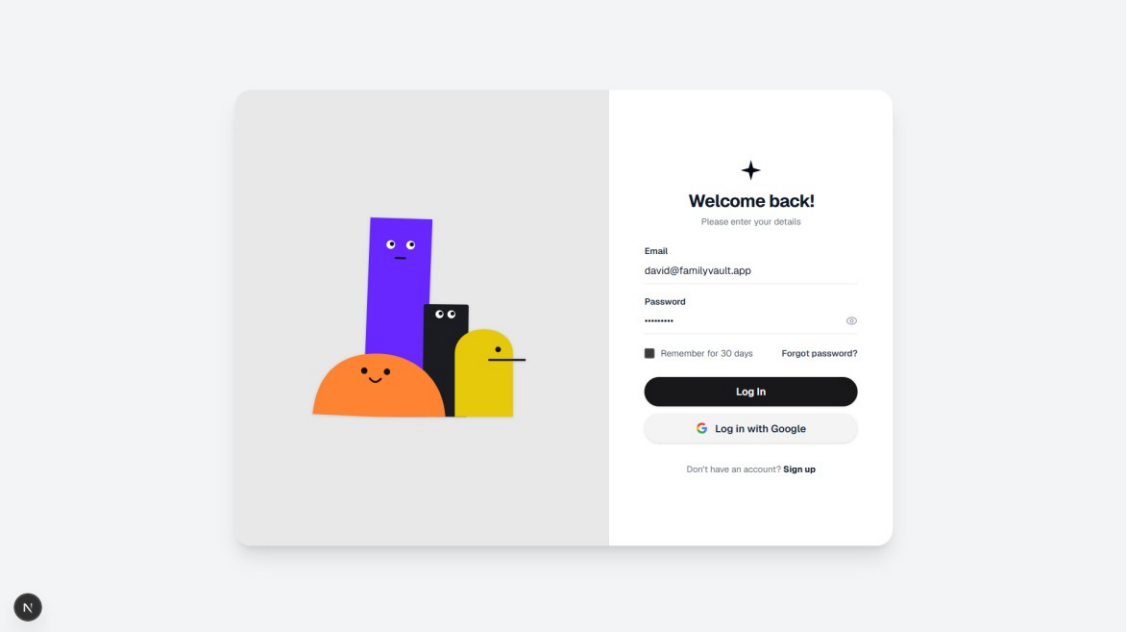


Fig A.1: Login Page

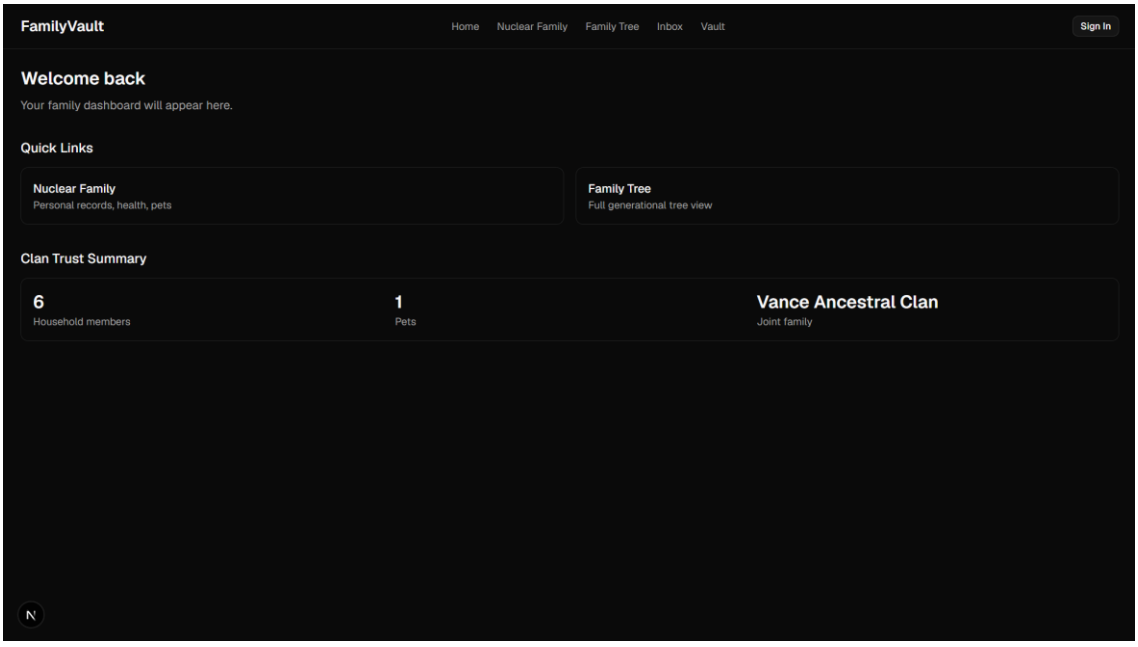


Fig A.2: Landing page

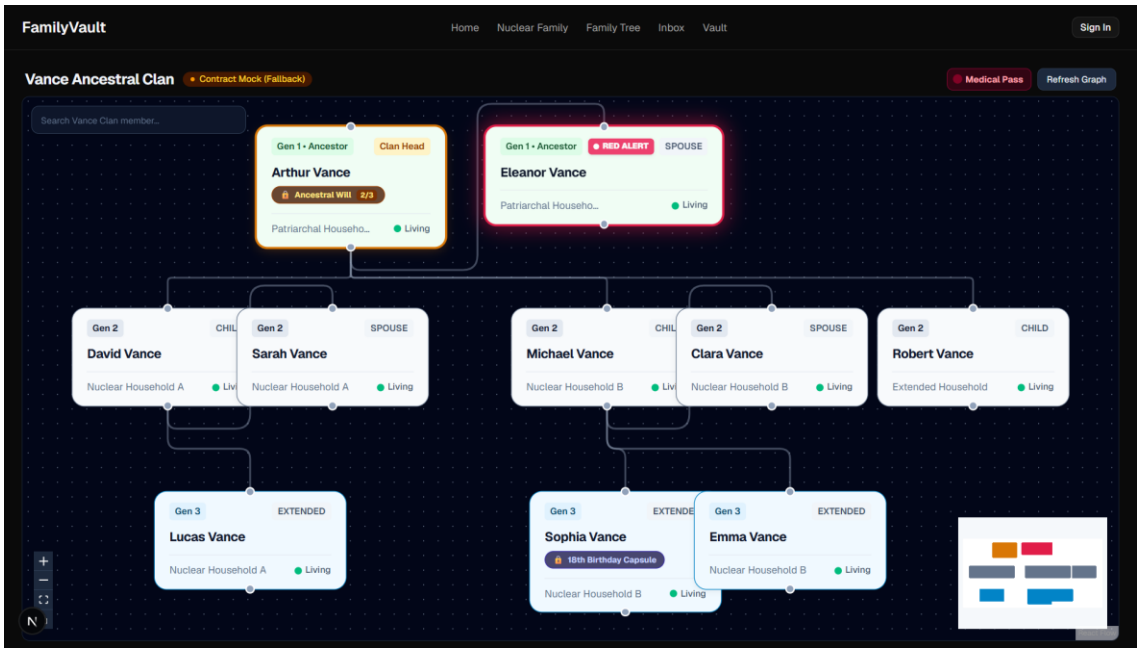


Fig A.3: Family tree canvas with kinship badges

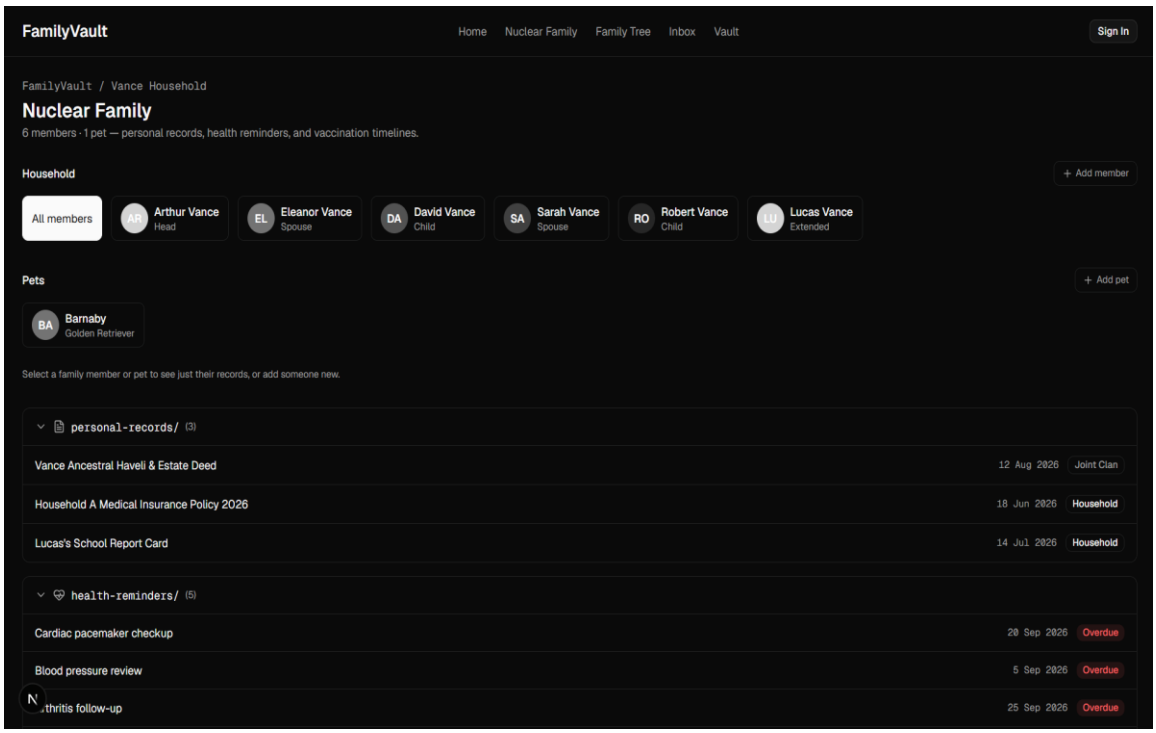


Fig A.3: Person drawer with gallery, health alerts and legacy letters

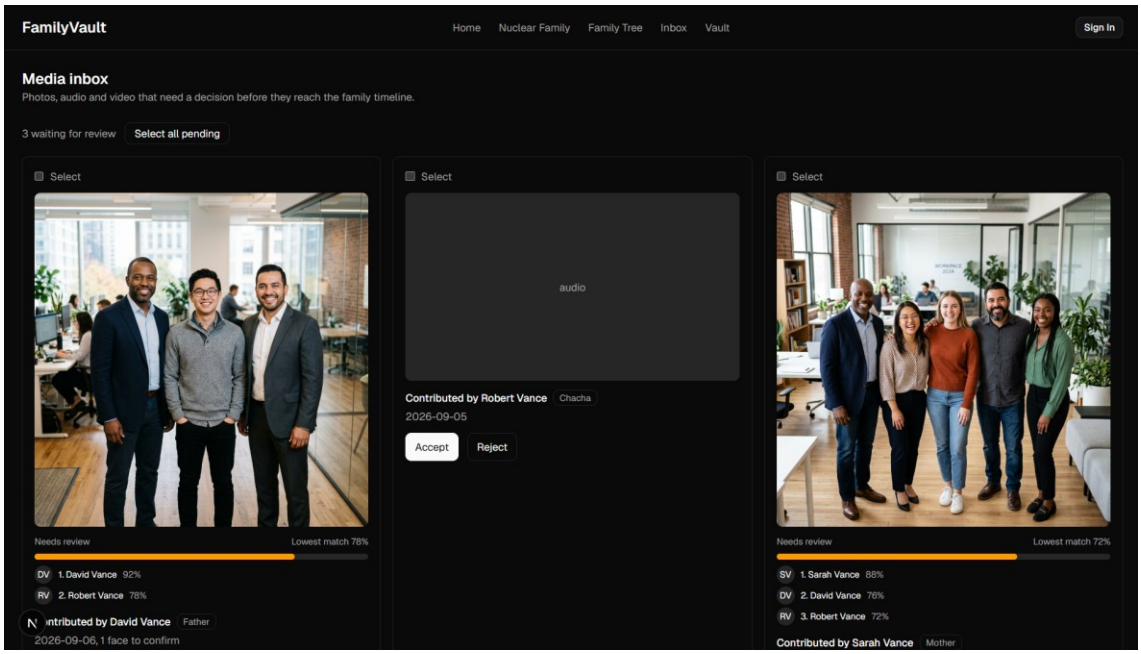


Fig A.4: Media inbox with Accept / Reject actions

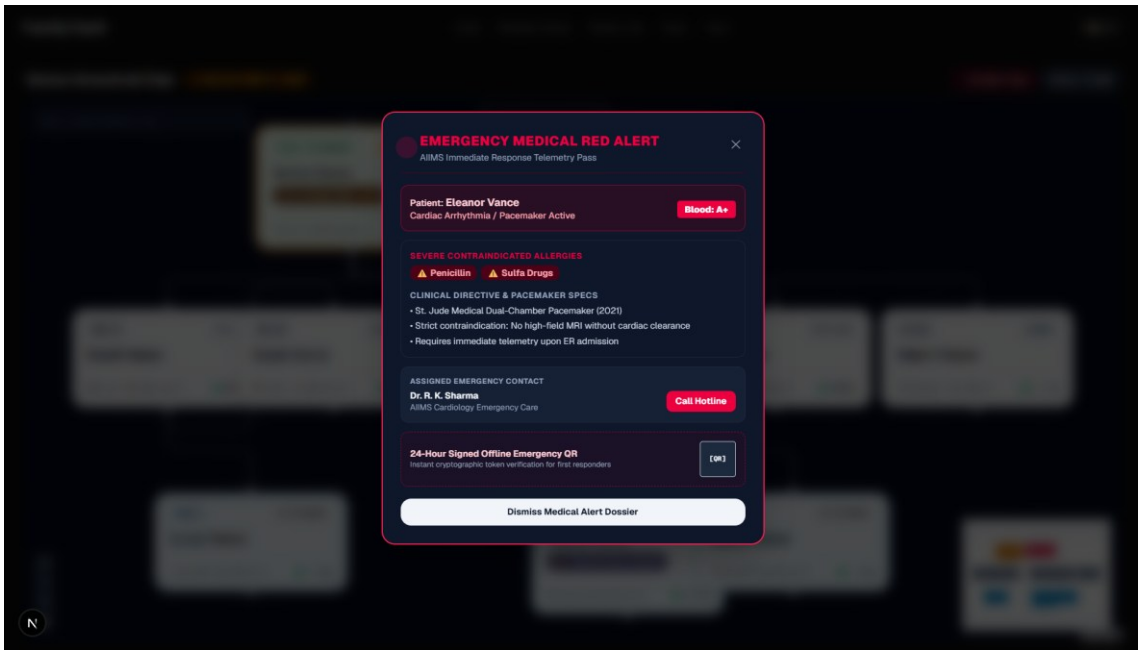


Fig A.5: Emergency Red Alert dossier output

APPENDIX B – CODING

Condensed, representative listings of the core logic are given below. The complete source code is maintained in the project's private GitHub repository.

Listing B.1: /contracts/MATCH_CONSTANTS.json

```
{
  "MATCH_THRESHOLD": 0.85,
  "EMBEDDING_DIMENSIONS": 512,
  "_note": "Single source of truth. Imported by /biometrics and by Ayush's test assertions."
}
```

Listing B.2: PostgreSQL schema excerpt and three-tier RLS policy

```
-- /contracts/schema.sql  (excerpt)
-- Rule: users.id is generated ONCE here and reused as Neo4j (:Person {id}).

CREATE TYPE visibility_tier AS ENUM ('JOINT_CLAN', 'NUCLEAR_HOUSEHOLD', 'PRIVATE_STRICT');
CREATE TYPE member_role     AS ENUM ('ADMIN', 'EDITOR', 'VIEWER');

CREATE TABLE joint_families (
  id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL
);

CREATE TABLE nuclear_families (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  joint_family_id UUID NOT NULL REFERENCES joint_families(id),
  name            TEXT NOT NULL
);

CREATE TABLE users (
  id                UUID PRIMARY KEY,             -- same value is used in Neo4j
  joint_family_id   UUID NOT NULL REFERENCES joint_families(id),
  nuclear_family_id UUID NOT NULL REFERENCES nuclear_families(id),
  full_name         TEXT NOT NULL,
  gender            CHAR(1) CHECK (gender IN ('M', 'F')),
  birth_date        DATE,
  role              member_role NOT NULL DEFAULT 'VIEWER'
);

CREATE TABLE documents (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  joint_family_id   UUID NOT NULL REFERENCES joint_families(id),
  nuclear_family_id UUID NOT NULL REFERENCES nuclear_families(id),
  owner_id          UUID NOT NULL REFERENCES users(id),
  title             TEXT NOT NULL,
  storage_key       TEXT NOT NULL,
  visibility        visibility_tier NOT NULL DEFAULT 'NUCLEAR_HOUSEHOLD',
  unlock_conditions JSONB                         -- e.g. {"not_before": "2044-05-01T00:00:00Z"}
  is_unlocked       BOOLEAN NOT NULL DEFAULT TRUE
);

-- Helper functions used by the policies
CREATE FUNCTION current_joint_family() RETURNS UUID
  LANGUAGE sql STABLE SECURITY DEFINER
  AS $$ SELECT joint_family_id FROM users WHERE id = auth.uid() $$;
```

```
CREATE FUNCTION current_nuclear_family() RETURNS UUID
  LANGUAGE sql STABLE SECURITY DEFINER
  AS $$ SELECT nuclear_family_id FROM users WHERE id = auth.uid() $$;

-- Three-tier Row Level Security
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;

CREATE POLICY documents_select ON documents FOR SELECT USING (
  (is_unlocked OR owner_id = auth.uid())
  AND (
       (visibility = 'JOINT_CLAN'        AND joint_family_id   = current_joint_family())
    OR (visibility = 'NUCLEAR_HOUSEHOLD' AND nuclear_family_id = current_nuclear_family())
    OR (visibility = 'PRIVATE_STRICT'    AND (owner_id = auth.uid()
         OR EXISTS (SELECT 1 FROM document_trustees t
                    WHERE t.document_id = documents.id AND t.trustee_id = auth.uid())))
  )
);

CREATE POLICY documents_insert ON documents FOR INSERT WITH CHECK (
  owner_id = auth.uid() AND nuclear_family_id = current_nuclear_family()
);
```

Listing B.3: Neo4j constraint, paradox guard and shortest-path query

```
// 1. Constraint: one :Person node per Postgres users.id
CREATE CONSTRAINT person_id_unique IF NOT EXISTS
FOR (p:Person) REQUIRE p.id IS UNIQUE;

// 2. Graph-paradox guard, run BEFORE creating a PARENT_OF edge.
//    Returns a row only if the edge would be impossible.
MATCH (p:Person {id: $parent_id}), (c:Person {id: $child_id})
WHERE p.id = c.id                                    // self-parenting
   OR c.birth_date <= p.birth_date                   // child not younger than parent
   OR EXISTS { (c)-[:PARENT_OF*1..]->(p) }           // cyclic ancestry
RETURN p.id AS parent_id, c.id AS child_id;

// 3. Shortest relationship path between two members (tenant-scoped)
MATCH (a:Person {id: $from_id, joint_family_id: $jf}),
      (b:Person {id: $to_id,   joint_family_id: $jf})
MATCH path = shortestPath((a)-[:PARENT_OF|SPOUSE_OF*..12]-(b))
RETURN [n IN nodes(path) | {id: n.id, gender: n.gender, birth_year: n.birth_year}] AS people,
       [r IN relationships(path) | type(r)] AS rel_types,
       [r IN relationships(path) | startNode(r).id] AS rel_starts;
```

Listing B.4: Mapping graph hops to a kinship title

```
from dataclasses import dataclass

@dataclass
class Hop:
    kind: str        # "UP" (to a parent), "DOWN" (to a child), "SPOUSE"
    gender: str      # gender of the person reached by this hop: "M" or "F"
    birth_year: int

def kinship_title(path: list[Hop]) -> tuple[str, str]:
    """Map an ego-to-target hop sequence to (title, lineage side)."""
    kinds = tuple(h.kind for h in path)

    if kinds == ("UP",):
        return ("Papa", "Paternal") if path[0].gender == "M" else ("Mummy", "Maternal")
```

```
    if kinds == ("UP", "UP", "DOWN"):            # sibling of a parent
        parent, _, target = path
        if parent.gender == "M":                 # father's side
            if target.gender == "F":
                return "Bua", "Paternal"
            elder = target.birth_year < parent.birth_year
            return ("Tauji" if elder else "Chacha"), "Paternal"
        # mother's side
        return ("Mama" if target.gender == "M" else "Masi"), "Maternal"

    return "Relative", "Unclassified"            # extended in later iterations
```

Listing B.5: Face embedding, cosine matching and routing

```
import json
import numpy as np
from insightface.app import FaceAnalysis

with open("contracts/MATCH_CONSTANTS.json") as f:
    MATCH_THRESHOLD = json.load(f)["MATCH_THRESHOLD"]     # 0.85, never hard-coded twice

face_app = FaceAnalysis(name="buffalo_l")
face_app.prepare(ctx_id=-1, det_size=(640, 640))          # ctx_id=-1 -> CPU inference

def embed_faces(image_bgr):
    """One L2-normalised 512-D embedding per detected face (supports group photos)."""
    return [face.normed_embedding for face in face_app.get(image_bgr)]

def cosine(a, b) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def route_media(media_type, image_bgr, gallery):
    """
    gallery: {person_uuid: reference_embedding}
    Returns ("AUTO_PUBLISH" | "INBOX", matches)
    """
    if media_type in ("video", "audio"):
        return "INBOX", []                                # always human-reviewed

    embeddings = embed_faces(image_bgr) if image_bgr is not None else []
    if not embeddings:
        return "INBOX", []                                # zero-face upload

    matches, every_face_confident = [], True
    for emb in embeddings:
        person_id, score = max(
            ((pid, cosine(emb, ref)) for pid, ref in gallery.items()),
            key=lambda item: item[1], default=(None, 0.0),
        )
        if score >= MATCH_THRESHOLD:                      # 0.850 passes, 0.849 does not
            matches.append({"person_id": person_id, "score": round(score, 3)})
        else:
            every_face_confident = False

    return ("AUTO_PUBLISH" if every_face_confident else "INBOX"), matches
```

Listing B.6: Emergency Red Alert dossier endpoint

```
from uuid import UUID
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel

router = APIRouter()

class EmergencyDossier(BaseModel):
    full_name: str
    blood_group: str | None
    allergies: list[str]
    vital_prescriptions: list[dict]
    emergency_contacts: list[dict]

@router.post("/api/v1/health/emergency-dossier/{user_id}", response_model=EmergencyDossier)
async def emergency_dossier(user_id: UUID, ctx=Depends(get_auth_context)):
    """Compile the 'Red Alert' summary. Row Level Security still applies,
    so a caller can only compile dossiers for members visible to them."""
    profile = await load_health_profile(ctx.db, user_id)       # runs under the caller's JWT
    if profile is None:
        raise HTTPException(status_code=404, detail="No health profile found")

    return EmergencyDossier(
        full_name=profile.full_name,
        blood_group=profile.blood_group,
        allergies=profile.allergies,
        vital_prescriptions=await load_vital_prescriptions(ctx.db, user_id),
        emergency_contacts=await load_emergency_contacts(ctx.db, user_id),
    )
```

Listing B.7: Tree canvas loading mock or live data

```
// frontend/components/TreeCanvas.tsx  (condensed)
"use client";
import { useEffect, useState } from "react";
import { ReactFlow, Background, Controls, type Node, type Edge } from "@xyflow/react";
import "@xyflow/react/dist/style.css";
import PersonNode from "./PersonNode";

const nodeTypes = { person: PersonNode };
// Contracts-first: start on the mock, switch to the live API via an env flag.
const TREE_URL =
  process.env.NEXT_PUBLIC_USE_MOCK === "true"
    ? "/contracts/graph-tree-response.json"
    : process.env.NEXT_PUBLIC_GRAPH_API + "/api/graph/tree";

export default function TreeCanvas({ onSelect }: { onSelect: (id: string) => void }) {
  const [nodes, setNodes] = useState<Node[]>([]);
  const [edges, setEdges] = useState<Edge[]>([]);

  useEffect(() => {
    fetch(TREE_URL)
      .then((r) => r.json())
      .then((tree) => {
        setNodes(tree.nodes.map((n: any) => ({
          id: n.id, type: "person", position: n.position,
          data: { name: n.name, badge: n.kinship_badge, role: n.role },
        })));
        setEdges(tree.edges.map((e: any) => ({ id: e.id, source: e.from, target: e.to })));
      });
  }, []);
```

```
  return (
    <ReactFlow nodes={nodes} edges={edges} nodeTypes={nodeTypes}
               minZoom={0.3} maxZoom={2} fitView
               onNodeClick={(_, node) => onSelect(node.id)}>
      <Background /><Controls />
    </ReactFlow>
  );
}
```

REFERENCES

- Beck, K. et al. (2001). Manifesto for Agile Software Development. (https://agilemanifesto.org)
- Cloudflare. Cloudflare R2 documentation. (https://developers.cloudflare.com/r2/)
- Conventional Commits. Conventional Commits Specification. (https://www.conventionalcommits.org)
- Deng, J., Guo, J., Xue, N. and Zafeiriou, S. (2019). ArcFace: Additive Angular Margin Loss for Deep Face Recognition. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4690–4699.
- Government of India (2023). The Digital Personal Data Protection Act, 2023.
- InsightFace Project. InsightFace: 2D and 3D face analysis. (https://github.com/deepinsight/insightface)
- Neo4j. Cypher Manual. (https://neo4j.com/docs/cypher-manual/current/)
- OWASP Foundation. OWASP Top Ten. (https://owasp.org/www-project-top-ten/)
- PostgreSQL Global Development Group. Row Security Policies. (https://www.postgresql.org/docs/current/ddl-rowsecurity.html)
- Robinson, I., Webber, J. and Eifrem, E. (2015). Graph Databases, 2nd ed. O'Reilly Media.
- Supabase. Row Level Security guide. (https://supabase.com/docs/guides/database/postgres/row-level-security)
- Tiangolo, S. FastAPI documentation. (https://fastapi.tiangolo.com)
- Vercel. Next.js documentation. (https://nextjs.org/docs)
- React Flow documentation. (https://reactflow.dev)